NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

Official Study Material • Java Programming & OOP (JAVA101) • Unit 4: Java Collections Framework & I/O Streams

Course Code:	JAVA101	**Regulation:**	R20 / R23 Autonomous
Subject:	Java Programming & OOP	**Unit:**	Unit 4 - Java Collections Framework & I/O Streams

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC STUDY MATERIAL: JAVA101 JAVA PROGRAMMING & OOP

UNIT 4: JAVA COLLECTIONS FRAMEWORK & I/O STREAMS

Regulation: R20 / R23 Autonomous Syllabus | Credits: 4

1. JAVA COLLECTIONS FRAMEWORK:

The Java Collections Framework provides standardized data structures and algorithms.

- Core Interfaces & Classes:

* List: Ordered collection, allows duplicates.

- ArrayList: Resizable array, fast random access O(1), slower insertions/deletions O(N).

- LinkedList: Doubly linked list, fast insertions/deletions O(1), slower search O(N).

* Set: Unordered collection, prohibits duplicate elements.

- HashSet: Backed by hash table, O(1) basic operations, unordered.

- TreeSet: Backed by Red-Black tree, sorted natural order, O(log N) operations.

* Map: Key-Value pairs, unique keys.

- HashMap: Hash table implementation, allows one null key, fast O(1).

- TreeMap: Red-Black tree sorted by key, O(log N).

* Queue: FIFO ordering (PriorityQueue, ArrayDeque).

* Iterator: Unified cursor to traverse elements (hasNext(), next(), remove()).

2. GENERICS & LAMBDA EXPRESSIONS:

- Generics: Provides compile-time type safety and eliminates explicit typecasting: List<String> list = new ArrayList<>();

- Lambda Expressions: Anonymous functions providing concise implementation of Functional Interfaces (interfaces with exactly one abstract method): (a, b) -> a + b.

3. FILE I/O STREAMS:

- Byte Streams: Read/write raw 8-bit bytes (images, audio, binaries).

* FileInputStream, FileOutputStream.

- Character Streams: Read/write 16-bit Unicode text characters.

* FileReader, FileWriter, BufferedReader, BufferedWriter.

- Try-With-Resources: Automatically closes resources implementing AutoCloseable without explicit finally block.

4. PRACTICAL CODE EXAMPLE:

```
// Java Program demonstrating HashMap, Collections, Lambdas, and Streams
import java.util.*;

public class CollectionsDemo {
public static void main(String[] args) {
Map<String, Integer> studentMarks = new HashMap<>();
studentMarks.put("Ananya", 92);
studentMarks.put("Karthik", 78);
studentMarks.put("Bhavani", 88);
studentMarks.put("Deepak", 65);

System.out.println("--- High Performing Students (>= 80) ---");
studentMarks.entrySet().stream()
.filter(entry -> entry.getValue() >= 80)
.sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
.forEach(e -> System.out.println(e.getKey() + ": " + e.getValue() + " marks"));
}
}
```

Output:

--- High Performing Students (>= 80) ---

Ananya: 92 marks

Bhavani: 88 marks

5. IMPORTANT EXAM QUESTIONS & ANSWERS:

Q1: Differentiate between ArrayList and LinkedList. When should each be used?

Answer: ArrayList uses a contiguous dynamic array, providing fast O(1) indexed retrieval but O(N) element shifts during insertions/deletions. LinkedList uses doubly-linked nodes, providing O(1) insertions/removals once node is located but O(N) random access. Use ArrayList for read-heavy operations and LinkedList for frequent insertions/removals.

Q2: Differentiate between Byte Streams and Character Streams in Java File I/O.

Answer: Byte streams (InputStream/OutputStream) operate on 8-bit bytes, ideal for binary files like images and compiled files. Character streams (Reader/Writer) operate on 16-bit Unicode characters with automatic character-set encoding, preventing international character corruption in text files.